

**NATIONAL INSTITUTE OF TECHNOLOGY AGARTALA**



**LINEAR CONTROL SYSTEMS LABORATORY PROJECT REPORT**

**PROJECT TITLE:**

**“FAN SPEED CONTROL BASED ON TEMPERATURE”**

**SUBMITTED BY:**

1. NETHAVATH KRISHNA - 23UEI150
2. BONAGA THULASIRAM - 23UEI166
3. THOTA GOWREESH - 23UEI164
4. ARE LEELA SAHITH - 23UEI161
5. RATHLAVATH NARESH - 23UEI167
6. SHANKAR KUMAR - 23UEI129

**GUIDED BY:**

**Dr. Priyanka Dey**

Assistant professor, Department of EIE

**Mr. Suraj Das**

**Date of submission: 10/11/2025**

## **Table of Contents:**

- 1)** Abstract
- 2)** Introduction
- 3)** Literature Survey
- 4)** Methodology
  - a)** Block Diagram
  - b)** Circuit Diagram
- 5)** Components and Data sheets
- 6)** Implementation
- 7)** Results and Discussion
- 8)** Conclusion
- 9)** References
- 10)** Appendices

## 1) Abstract:

This project focuses on designing and implementing a temperature-controlled fan system using an Arduino Nano, a DHT11 temperature sensor, and a transistor-based driver circuit. The primary objective is to automatically regulate the cooling fan's speed based on real-time temperature variations, demonstrating the practical application of linear control system concepts. The system continuously monitors the ambient temperature using the DHT11 sensor and compares it with a predefined setpoint. A proportional control strategy is implemented in the microcontroller, where the error between the measured temperature and the setpoint is converted into a PWM signal. This PWM output adjusts the fan speed smoothly, ensuring effective thermal management. The transistor acts as an interface to safely drive the 5V DC fan based on the PWM control signal.

The project highlights key linear control elements such as feedback, error correction, and proportional control response. Experimental results show that the fan speed increases linearly with temperature rise, providing stable and efficient cooling. Overall, the system is simple, low-cost, and demonstrates how basic control system principles can be applied to real-world automation tasks.

## **2) Introduction**

Temperature regulation is an essential requirement in many engineering applications, and linear control systems provide an effective framework for achieving stable and automatic cooling. This project presents a temperature-controlled fan system designed using the principles of linear control, where the fan speed is continuously adjusted according to the measured temperature. By implementing a simple feedback mechanism, the system maintains thermal stability while responding smoothly to temperature variations.

The project utilizes an Arduino Nano microcontroller, a DHT11 temperature sensor, and a transistor-based driver to control a 5V cooling fan. The central problem addressed is the need for efficient temperature-dependent fan regulation, avoiding the limitations of simple ON–OFF control methods. Instead, proportional control is used to vary the fan speed linearly with temperature, ensuring better performance and reduced power consumption.

This work demonstrates key concepts of linear control systems—such as feedback, error correction, and proportional response—through a practical and low-cost implementation, making it relevant for students, industries, and embedded system applications.

### **3) Literature Survey:**

Research on temperature-controlled fan systems has evolved steadily over the past decade. Early studies around 2013 demonstrated that using PWM to vary fan speed provides smoother operation and better efficiency compared to simple ON/OFF control. By 2017, student and academic projects had begun implementing microcontroller-based fan controllers using basic sensors and transistor drivers, proving that automatic temperature regulation can be achieved with low-cost electronics.

Between 2019 and 2022, several Arduino-based designs showed that digital temperature sensors such as the DHT11 can be effectively used for closed-loop thermal control despite their limited sampling rate. These works also highlighted the importance of using feedback to maintain stable operation. More recent developments around 2023–2024 explored proportional, PI, and PID controllers for thermal systems, concluding that proportional control alone is often sufficient for small cooling applications like DC fan regulation.

Overall, past research consistently shows that PWM-based fan control combined with a simple feedback loop offers a reliable, efficient, and practical approach for temperature-dependent speed control. This project builds upon these findings and demonstrates a similar linear control strategy using Arduino Nano, DHT11, and a transistor-driven fan.

#### 4) Methodology

##### i) Circuit Diagram:

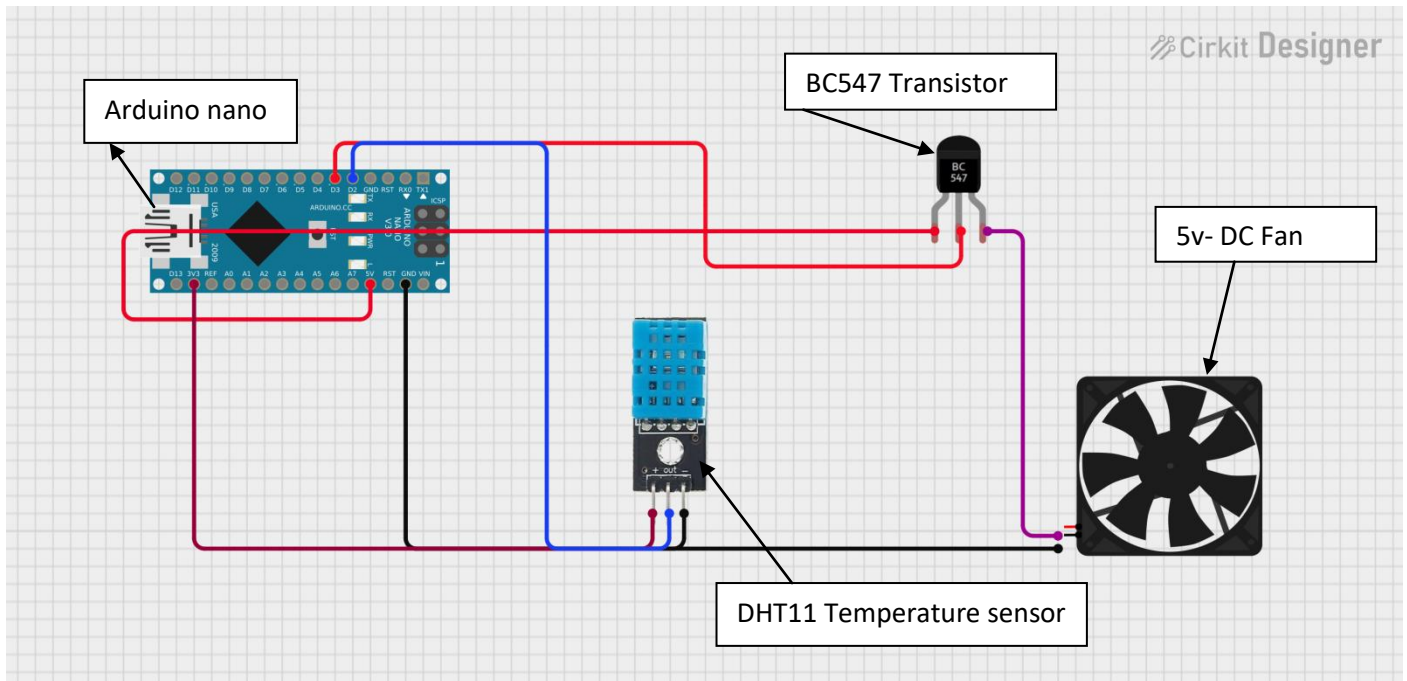


Fig 1 : Circuit Diagram

##### ii) Block Diagram

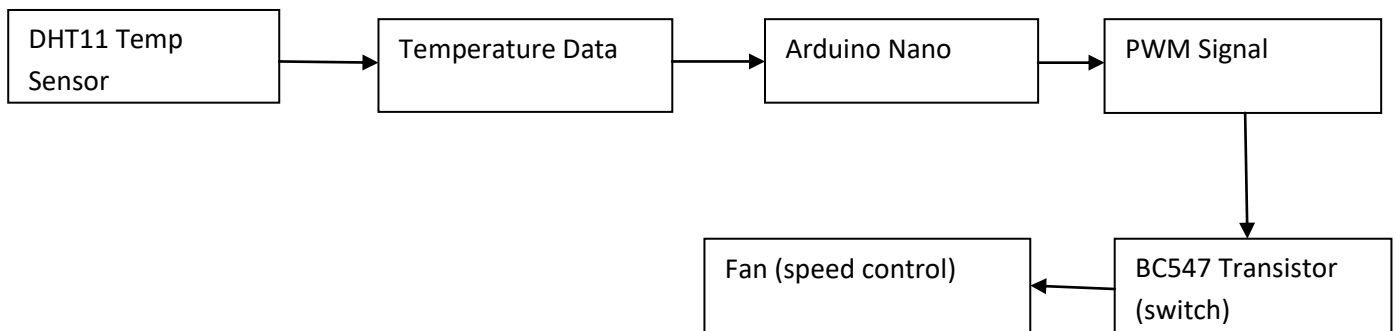


Fig 2 : Block Diagram

## 5) Components and Data sheets

| S.no | Name of Apparatus  | Specifications | Quantity    |
|------|--------------------|----------------|-------------|
| 1.   | Arduino            | Nano           | 1           |
| 2.   | DC fan             | 5v             | 1           |
| 3.   | Transistor         | BC547          | 1           |
| 4.   | Temperature Sensor | DHT11          | 1           |
| 5.   | Bread Board        | -              | 1           |
| 6.   | Jumper wires       | -              | As required |

## 6) Implementation

### Gather Components:

Arduino Nano, DHT11 temperature sensor, 5V DC fan, BC547 NPN transistor, Jumper wires, breadboard, USB cable

### Wiring the Circuit:

-DHT11 connections:

VCC to 5V on Arduino Nano

GND to GND on Arduino Nano

OUT (signal) to a digital pin on Arduino (e.g., D2)

-Fan connections:

Positive terminal to Arduino 5V

Negative terminal to Collector of BC547

-BC547 connections:

Base directly to a digital pin of Arduino (e.g., D3).

### Programming the Arduino

Connect Arduino to PC and set up the Arduino IDE.

Install the DHT sensor library.

Write/upload code that:

Reads temperature and humidity from the DHT11. When temperature rises above your setpoint, sets the chosen digital pin HIGH (to activate the fan) or outputs PWM for speed control. Otherwise, sets the digital pin LOW (fan off).

### Testing:

Plug in power and review performance using the Serial Monitor. Check that the fan responds as expected when the temperature crosses your set threshold. If not, debug wiring/code.

### Mounting and Final Build:

Once verified, secure components (breadboard or PCB, housing, etc.). Arrange the DHT11 sensor for proper ambient exposure.

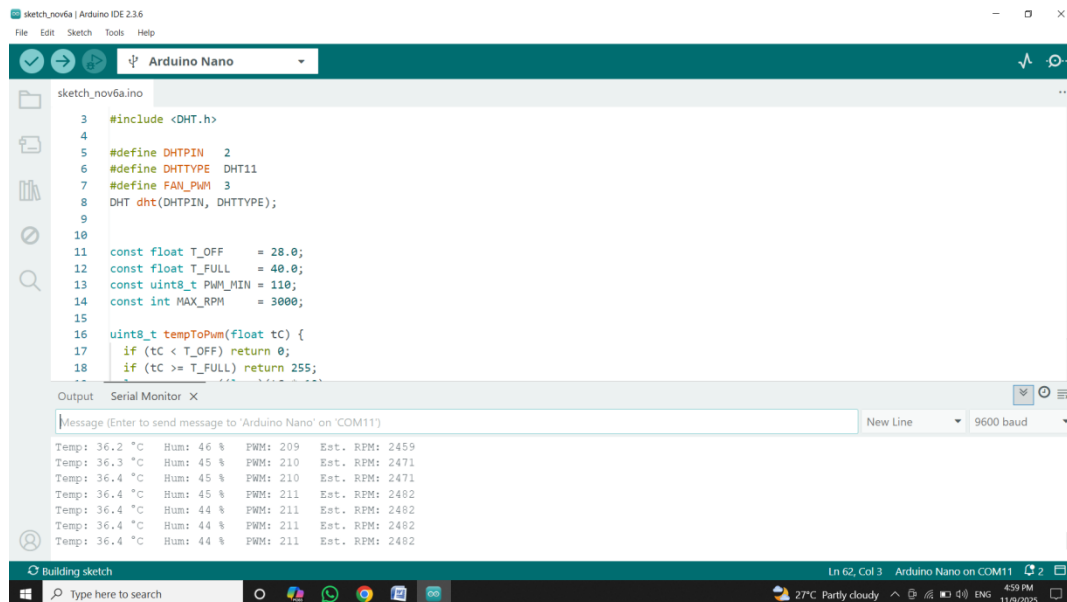


## 7) Results

The system was tested by gradually increasing the ambient temperature near the **DHT11 sensor** using gentle warm air (from a lighter or body heat).

The Arduino Nano successfully read the temperature and humidity values and adjusted the **fan speed (PWM output)** accordingly.

The Serial Monitor displayed the corresponding **Temperature (°C)**, **Humidity (%)**, **PWM value**, and **Estimated RPM**.



The screenshot shows the Arduino IDE interface. The sketch editor displays the code for 'sketch\_nov6a.ino'. The code includes the DHT library, defines pins and sensor type, and sets constants for temperature thresholds and PWM values. The Serial Monitor shows the output of the sketch, displaying temperature, humidity, PWM, and estimated RPM values.

```
3 #include <DHT.h>
4
5 #define DHTPIN 2
6 #define DHTTYPE DHT11
7 #define FAN_PWM 3
8 DHT dht(DHTPIN, DHTTYPE);
9
10
11 const float T_OFF = 28.0;
12 const float T_FULL = 40.0;
13 const uint8_t PWM_MIN = 110;
14 const int MAX_RPM = 3000;
15
16 uint8_t tempToPwm(float tc) {
17   if (tc < T_OFF) return 0;
18   if (tc >= T_FULL) return 255;
```

Serial Monitor Output:

| Temp: 36.2 °C | Hum: 46 % | PWM: 209 | Est. RPM: 2459 |
|---------------|-----------|----------|----------------|
| Temp: 36.3 °C | Hum: 45 % | PWM: 210 | Est. RPM: 2471 |
| Temp: 36.4 °C | Hum: 45 % | PWM: 210 | Est. RPM: 2471 |
| Temp: 36.4 °C | Hum: 45 % | PWM: 211 | Est. RPM: 2482 |
| Temp: 36.4 °C | Hum: 44 % | PWM: 211 | Est. RPM: 2482 |
| Temp: 36.4 °C | Hum: 44 % | PWM: 211 | Est. RPM: 2482 |
| Temp: 36.4 °C | Hum: 44 % | PWM: 211 | Est. RPM: 2482 |

### Result Summary

- At **room temperature (~27°C)** → assuming fan remains OFF.
- As temperature increases above **28°C**, the Arduino generates PWM signals with increasing duty cycle.
- The **PWM value and fan speed** increase proportionally with temperature.
- At **40°C and above**, the fan operates at full speed (PWM = 255).
- The **estimated RPM** values increase accordingly, confirming correct proportional control.

Hence, the automatic fan speed control system successfully varies fan speed with temperature using the DHT11 sensor and Arduino Nano.

## 8) Conclusion

The project “Automatic Fan Speed Control using DHT11 Sensor and Arduino Nano” successfully detects ambient temperature and adjusts the fan speed automatically based on temperature changes.

- The **DHT11 sensor** accurately measures temperature and humidity.
- The **Arduino Nano** processes the temperature data and generates a **PWM signal** to control the fan.
- At lower temperatures, the fan remains OFF or runs slowly.
- As the temperature rises, the **PWM duty cycle increases**, causing the fan to spin faster.
- The fan speed variation was clearly visible in the Serial Monitor output.
- The system works as a **closed-loop control system**, maintaining comfort and saving power automatically.

## 9) References

1. **Adafruit Industries.**  
*DHT11 Temperature and Humidity Sensor Datasheet* — <https://cdn-shop.adafruit.com/datasheets/DHT11.pdf>  
— (For temperature range, accuracy, and timing specifications of DHT11.)
2. **Arduino Official Documentation.**  
*Arduino Reference – analogWrite(), digitalWrite(), and PWM functions* — <https://www.arduino.cc/reference/en>  
— (For detailed function explanations used in code.)
3. **Robu.in.**  
*Change Fan Speed with Temperature using Arduino* — <https://robu.in/change-fan-speed-with-temperature/>  
— (Original project concept adapted in this experiment.)
4. **Datasheet: BC547 NPN Transistor.**  
*Philips Semiconductors / Nexperia, BC547/BC548/BC549 Series.*  
— (For transistor current ratings and pin configuration used in fan driver.)

10) Appendices

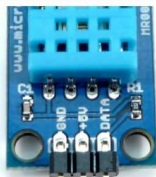


Fig. 1 - Signals

CHARACTERISTICS

|                           |                                    |
|---------------------------|------------------------------------|
| Supply voltage            | +5V                                |
| Supply current (running)  | 0.5mA typ. (2.5mA max.)            |
| Supply current (stand-by) | 100uA typ. (150uA max.)            |
| Temperature range         | 0 / +50°C ±2°C                     |
| Humidity range            | 20-90%RH ±5%RH                     |
| Interface                 | Digital                            |
| Dimensions                | 1.05" x 0.7" (connectors excluded) |
| Weight                    | 0.1 oz (2.7g)                      |

Tab.2 - Characteristics

**BC546/547/548/549/550**

**Switching and Applications**

- High Voltage: BC546,  $V_{CE0}=65V$
- Low Noise: BC549, BC550
- Complement to BC556 ... BC560

**NPN Epitaxial Silicon Transistor**

**Absolute Maximum Ratings**  $T_a=25^{\circ}C$  unless otherwise noted

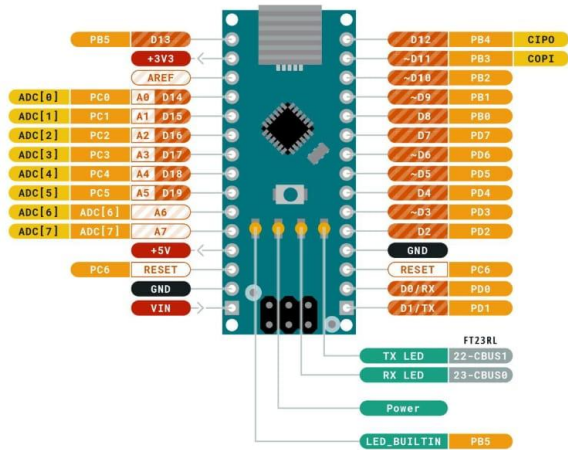
| Symbol    | Parameter                   | Value                                       | Units       |
|-----------|-----------------------------|---------------------------------------------|-------------|
| $V_{CB0}$ | Collector-Base Voltage      | BC546: 80<br>BC547/550: 50<br>BC548/549: 30 | V           |
| $V_{CE0}$ | Collector-Emitter Voltage   | BC546: 65<br>BC547/550: 45<br>BC548/549: 30 | V           |
| $V_{EB0}$ | Emitter-Base Voltage        | BC546/547: 6<br>BC548/549/550: 5            | V           |
| $I_C$     | Collector Current (DC)      | 100                                         | mA          |
| $P_C$     | Collector Power Dissipation | 500                                         | mW          |
| $T_J$     | Junction Temperature        | 150                                         | $^{\circ}C$ |
| $T_{STG}$ | Storage Temperature         | -65 - 150                                   | $^{\circ}C$ |

**Electrical Characteristics**

TO-92

1. Collector 2. Base 3. Emitter

547/548/549/550



### 5.1 Analog

| Pin | Function | Type   | Description          |
|-----|----------|--------|----------------------|
| 1   | +3V3     | Power  | 5V USB Power         |
| 2   | A0       | Analog | Analog input 0 /GPIO |
| 3   | A1       | Analog | Analog input 1 /GPIO |
| 4   | A2       | Analog | Analog input 2 /GPIO |
| 5   | A3       | Analog | Analog input 3 /GPIO |
| 6   | A4       | Analog | Analog input 4 /GPIO |
| 7   | A5       | Analog | Analog input 5 /GPIO |
| 8   | A6       | Analog | Analog input 6 /GPIO |
| 9   | A7       | Analog | Analog input 7 /GPIO |
| 10  | +5V      | Power  | +5V Power Rail       |
| 11  | Reset    | Reset  | Reset                |
| 12  | GND      | Power  | Ground               |
| 12  | VIN      | Power  | Voltage Input        |

### 5.2 Digital

| Pin | Function | Type    | Description            |
|-----|----------|---------|------------------------|
| 1   | D1/TX1   | Digital | Digital Input 1 /GPIO  |
| 2   | D0/RX0   | Digital | Digital Input 0 /GPIO  |
| 3   | D2       | Digital | Digital Input 2 /GPIO  |
| 4   | D3       | Digital | Digital Input 3 /GPIO  |
| 5   | D4       | Digital | Digital Input 4 /GPIO  |
| 6   | D5       | Digital | Digital Input 5 /GPIO  |
| 7   | D6       | Digital | Digital Input 6 /GPIO  |
| 8   | D7       | Digital | Digital Input 7 /GPIO  |
| 9   | D8       | Digital | Digital Input 8 /GPIO  |
| 10  | D9       | Digital | Digital Input 9 /GPIO  |
| 11  | D10      | Digital | Digital Input 10 /GPIO |
| 12  | D11      | Digital | Digital Input 11 /GPIO |
| 13  | D12      | Digital | Digital Input 12 /GPIO |
| 14  | D13      | Digital | Digital Input 13 /GPIO |
| 15  | Reset    | Reset   | Reset                  |
| 16  | GND      | Power   | Ground                 |

## Code:

```
#include <Arduino.h>
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11
#define FAN_PWM 3
DHT dht(DHTPIN, DHTTYPE);

const float T_OFF = 28.0;
const float T_FULL = 40.0;
const uint8_t PWM_MIN = 110;
const int MAX_RPM = 3000;

uint8_t tempToPwm(float tC) {
    if (tC < T_OFF) return 0;
    if (tC >= T_FULL) return 255;
    long pwm = map((long)(tC * 10),
        (long)(T_OFF * 10), (long)(T_FULL * 10),
        PWM_MIN, 255);
    if (pwm < 0) pwm = 0;
    if (pwm > 255) pwm = 255;
    return (uint8_t)pwm;
}

void setup() {
    Serial.begin(9600);
    pinMode(FAN_PWM, OUTPUT);
    analogWrite(FAN_PWM, 0);
    dht.begin();

    Serial.println(F("-----"));
    Serial.println(F("Automatic Fan Speed Control using DHT11 & Arduino"));
    Serial.println(F("Pins: DHT11->D2, Fan PWM->D3"));
    Serial.println(F("Output Columns: Temp (°C), Hum (%), PWM, Est. RPM"));
    Serial.println(F("-----"));
}

void loop() {
    delay(2000);

    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    if (isnan(temperature) || isnan(humidity)) {
        Serial.println(F("DHT11 read error! Check wiring."));
        return;
    }

    static float tFiltered = NAN;
    if (isnan(tFiltered)) tFiltered = temperature;
    tFiltered = 0.7 * tFiltered + 0.3 * temperature;

    uint8_t pwmValue = tempToPwm(tFiltered);
    analogWrite(FAN_PWM, pwmValue);

    float estimatedRPM = (pwmValue / 255.0) * MAX_RPM;

    Serial.print("Temp: ");
    Serial.print(tFiltered, 1);
    Serial.print(" °C Hum: ");
    Serial.print(humidity, 0);
    Serial.print(" % PWM: ");
    Serial.print(pwmValue);
    Serial.print(" Est. RPM: ");
    Serial.println(estimatedRPM, 0);
}
```

**Thank you**

